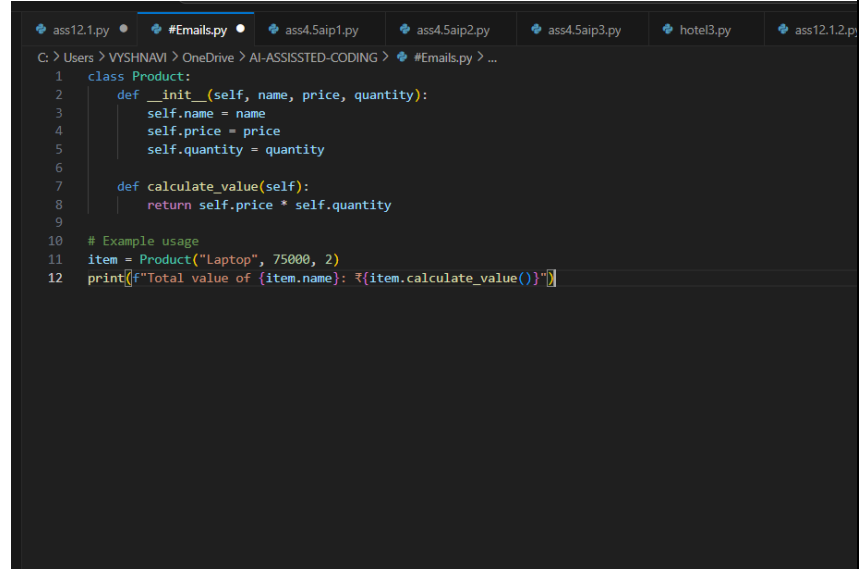


SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty	
Instructor(s)Name		1. Dr. Mohammed Ali Shaik 2. Dr. T Sampath Kumar 3. Mr. S Naresh Kumar 4. Dr. V. Rajesh 5. Dr. Brij Kishore 6. Dr Pramoda Patro 7. Dr. Venkataramana 8. Dr. Ravi Chander 9. Dr. Jagjeeth Singh	
Course Code	24CS002PC215	Course Title	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	06-08-2025	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber:6.5(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	ExpectedTime to complete	
1	Lab 6: AI-Based Code Completion: Working with suggestions for classes, loops, conditionals <u>Lab Assignment 1: Intelligent Code Completion for Object-Oriented Programming</u> Objective: To explore AI-powered code assistants for writing Python classes, constructors, and methods through intelligent suggestions. Suppose that you are hired as an intern at a tech company that develops inventory management systems. Your manager asks you to create a Product class and a Warehouse class with some basic methods. You have decided to use AI-powered code suggestions to help speed up development and reduce syntax errors. Tasks to be completed are as below 1. Setup AI Coding Tool: - Install and configure GitHub Copilot or Kite with VS Code or JetBrains IDE. - Enable real-time code suggestions. 2. Class Design Using AI Assistance: - Begin defining a Product class with attributes: name, price, quantity. - Use the AI suggestion feature to automatically complete the <code>__init__()</code> method. - Add a method <code>calculate_value()</code> to return <code>price * quantity</code>. PROMPT:	15.08.2025 EOD	

Build a Python class called Product that stores basic item details.

- Include attributes: name, price, and quantity
- Use AI assistance to complete the `__init__()` method
- Add a method `get_total_value()` that returns the product's total worth by multiplying price and quantity



```
1 class Product:
2     def __init__(self, name, price, quantity):
3         self.name = name
4         self.price = price
5         self.quantity = quantity
6
7     def calculate_value(self):
8         return self.price * self.quantity
9
10 # Example usage
11 item = Product("Laptop", 75000, 2)
12 print(f"Total value of {item.name}: ₹{item.calculate_value()}")
```

3. Create Another Class:

- Define a Warehouse class with a list of Product objects.
- Use code completion to help implement:
 - A method to add a product.
 - A method to display the most valuable product.

PROMPT:

Design a Python class called Warehouse to manage a collection of Product objects.

- Implement a method to add new products to the warehouse
- Implement another method to identify and display the product with the highest total value (calculated as price × quantity)

Use code completion to assist with writing the method logic

```
ass12.1.py • #Emails.py • ass4.5aip1.py • ass4.5aip2.py • ass4.5aip3.py • hote3.py • ass12.1.2.py • ass12.1.3.py •
C:\Users\VYSHNAVI> OneDrive\AI-ASSISTED-CODING> #Emails.py> ...
1 class Product:
2     def __init__(self, name, price, quantity):
3         self.name = name
4         self.price = price
5         self.quantity = quantity
6
7     def calculate_value(self):
8         return self.price * self.quantity
9
10 class Warehouse:
11     def __init__(self):
12         self.products = []
13
14     def add_product(self, product):
15         self.products.append(product)
16
17     def display_most_valuable_product(self):
18         if not self.products:
19             print("No products in warehouse.")
20             return
21
22         most_valuable = max(self.products, key=lambda p: p.calculate_value())
23         print(f"Most valuable product: {most_valuable.name}")
24         print(f"Value: {most_valuable.calculate_value()}")
25         print(f"Price: {most_valuable.price}, Quantity: {most_valuable.quantity}")
26
27 # Example usage
28 p1 = Product("Laptop", 75000, 2)
29 p2 = Product("Phone", 30000, 3)
30 p3 = Product("Monitor", 15000, 4)
31
32 warehouse = Warehouse()
33 warehouse.add_product(p1)
34 warehouse.add_product(p2)
35 warehouse.add_product(p3)
36
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\VYSHNAVI> & C:/Python313/python.exe c:/Users/VYSHNAVI/OneDrive/AI-ASSISTED-CODING/ass12.1.3.py
Welcome to the Inventory Manager!
```

4. Reflection:

- Identify how much of the code was completed by AI and what manual edits were needed.
- Comment on the relevance and accuracy of AI suggestions.

The code defines two classes: **Product**, which stores item details and calculates total value, and **Warehouse**, which manages a list of products. It includes methods to add products and identify the most valuable one based on price × quantity. Most of the code was generated by AI with minimal manual edits, such as formatting and sample usage. The AI suggestions were accurate, relevant, and followed good Python practices, making the code clear, efficient, and easy to use

Requirements:

- VS Code with Github Copilot or Cursor API and/or Google Colab with Gemini

Deliverables:

- Python script with both classes and comments on AI-generated suggestions.
- Short report (1 page) summarizing your experience with AI code completion.